

UIDE

LÓGICA DE PROGRAMACIÓN

Alumno: Andrade Espinoza Rodrigo Alejandro
Fecha: 20/08/2026

Contenido

Desafío: MiniTienda — registro y análisis de ventas	2
1. Explicación del algoritmo	2
2. Cumplimiento de requisitos	3
3. Capturas de ejecución	4
3.1 Importación de librerías	4
3.2 Catálogo inicial	4
3.3 Reto A: agregar producto nuevo	4
3.4 Registro de 12 ventas de prueba (incluye Reto C)	5
3.5 Reto D: producto_id inexistente y cantidad inválida	5
3.6 Guardar ventas.csv con Pandas	6
3.7 Leer ventas.csv de vuelta	7
3.8 Manejo de error: archivo inexistente	7
3.9 Ingresos por producto (Pandas groupby)	8
3.10 Métricas con NumPy	8
3.11 Gráfico de ingresos (Reto B)	8
3.12 Contenido de log.txt	9
4. Captura del gráfico: ingresos por producto	11
5. Respuestas a la asignación	12
¿Qué parte la hice con Pandas? ¿Qué parte con NumPy?	12
¿Dónde usé try/except y por qué?	12
¿Qué estructuras son tuplas, listas y diccionarios en el código?	12

Desafío: MiniTienda — registro y análisis de ventas

Este documento reúne la evidencia de desarrollo del programa MiniTienda: las capturas de ejecución con sus entradas y salidas, una explicación de cómo funciona el algoritmo y el detalle de cómo se cumplió cada uno de los requisitos pedidos en la consigna.

1. Explicación del algoritmo

Para el catálogo de productos usé una lista de tuplas, donde cada tupla guarda el id, el nombre y la categoría de un producto. Los precios y el stock los manejo aparte, en dos diccionarios independientes indexados por el id del producto, porque necesito poder consultarlos y actualizarlos rápido cada vez que se registra una venta.

El registro de una venta pasa por varias validaciones dentro de la función `registrar_venta()`. Primero se busca el producto en el catálogo recorriendo la lista con un bucle `for`; si el id no aparece, se lanza un error y, además, ese intento fallido queda anotado en `log.txt`, que es el Reto D. Después se valida que la cantidad pedida sea mayor a cero y que no supere el stock disponible. Si la cantidad es igual o mayor a diez unidades, se aplica automáticamente un cinco por ciento de descuento mediante una simple condición `if`, que es el Reto C. Cada venta que se completa con éxito se agrega como un diccionario a la lista `ventas_buffer`.

El menú del programa funciona con un bucle `while True` que se mantiene activo hasta que el usuario escoge la opción 0, momento en el que se ejecuta un `break` para salir. Dentro del menú uso una estructura `if/elif/else` para dirigir cada opción y un `continue` cuando el usuario ingresa algo que no está entre las opciones válidas. Para registrar una venta uso un bloque `try/except/else/finally` completo: el `try` intenta registrar la venta, el `except` atrapa los errores de producto inexistente o de cantidad o stock inválidos, el `else` se ejecuta solo si todo salió bien, y el `finally` siempre imprime un mensaje de cierre, haya habido error o no.

Para guardar y leer los datos uso Pandas: la función `guardar_ventas_csv()` convierte la lista `ventas_buffer` en un `DataFrame` y lo exporta con `to_csv()`, mientras que `leer_ventas_csv()` usa `pd.read_csv()` dentro de un `try/except` que atrapa el error si el archivo todavía no existe. Las métricas del negocio (ingreso total, promedio y desviación estándar) las calculo con NumPy, usando `np.sum`, `np.mean` y `np.std` sobre arreglos contruidos a partir de las columnas del `DataFrame`; la división para el promedio de unidades por venta está protegida con un `try/except` por si en algún momento no hubiera ventas registradas. Por último, `ingresos_por_producto()` agrupa los datos con `df.groupby('producto')['total'].sum()`, y esa información es la que grafico con Matplotlib y exporto como PNG con `plt.savefig()`.

2. Cumplimiento de requisitos

Requisito	Cómo se cumple
Tuplas: catálogo de productos	catalogo es una lista de tuplas (id, nombre, categoría).
Diccionarios: precios y stock	Diccionarios precios y stock, indexados por id_producto.
Listas: buffer de ventas / IDs	ventas_buffer (lista de diccionarios); id_venta autoincremental.
Funciones: todo modular	Cada tarea vive en su propia función: mostrar_catalogo, registrar_venta, guardar/leer CSV, métricas, graficar, menú.
Errores controlados	try/except para input inválido, archivo inexistente y validaciones de negocio.
Archivos: ventas.csv + log.txt	guardar_ventas_csv() y escribir_log() generan ambos archivos.
Pandas: DataFrame + groupby + CSV	pd.DataFrame(ventas_buffer), groupby('producto'), to_csv/read_csv.
NumPy: mean/std/sum	calcular_metricas() usa np.sum, np.mean, np.std sobre arreglos.
Matplotlib: gráfico de barras	graficar_ingresos() genera el bar chart de ingresos por producto.
Control y bucles completos	if/elif/else, for, while, break, continue, try/except/else/finally.
Reto A: agregar producto / actualizar precio y stock	Función agregar_producto() y opción 7 del menú.
Reto B: exportar gráfico a PNG	Opción 6 del menú, con plt.savefig('ingresos.png').
Reto C: descuento por cantidad ≥ 10	if cantidad ≥ 10 : descuento_pct = 0.05, dentro de registrar_venta().
Reto D: log de producto_id inexistente	escribir_log() anota el intento fallido antes de lanzar el error.

3. Capturas de ejecución

A continuación se muestra, celda por celda, el código que ejecuté en el notebook (MiniTienda.ipynb) junto con la salida real que obtuve en consola, como evidencia de que el programa funciona.

3.1 Importación de librerías

Código:

```
import os
from datetime import datetime

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

print("Librerías importadas correctamente:", pd.__version__, np.__version__)
```

Salida obtenida:

```
Librerías importadas correctamente: 3.0.2 2.4.4
```

3.2 Catálogo inicial

Código:

```
print(">>> Catálogo inicial")
mostrar_catalogo()
```

Salida obtenida:

```
>>> Catálogo inicial

--- CATÁLOGO DE PRODUCTOS ---
ID  Nombre      Categoría  Precio  Stock
1   Laptop      Cómputo   650.00  10
2   Mouse       Accesorios 15.50   50
3   Teclado     Accesorios 25.00   40
4   Monitor     Cómputo   180.00  15
5   Audífonos   Accesorios 35.00   30
```

3.3 Reto A: agregar producto nuevo

Código:

```
print(">>> Reto A: agregar producto nuevo")
agregar_producto(6, "Webcam", "Accesorios", 45.00, 20)
mostrar_catalogo()
```

Salida obtenida:

```
>>> Reto A: agregar producto nuevo
Producto 'Webcam' agregado al catálogo.
```

```
--- CATÁLOGO DE PRODUCTOS ---
ID  Nombre          Categoría    Precio   Stock
1   Laptop          Cómputo     650.00   10
2   Mouse           Accesorios   15.50    50
3   Teclado         Accesorios   25.00    40
4   Monitor         Cómputo     180.00   15
5   Audífonos       Accesorios   35.00    30
6   Webcam          Accesorios   45.00    20
```

3.4 Registro de 12 ventas de prueba (incluye Reto C)

Código:

```
print(">>> Registrando ventas de prueba (incluye descuento Reto C)")
ventas_prueba = [
    (1, 1), (2, 5), (3, 12), (4, 2), (5, 3),
    (2, 10), (1, 2), (6, 4), (3, 3), (4, 1),
    (5, 15), (2, 3),
]

for pid, cantidad in ventas_prueba:
    try:
        venta = registrar_venta(pid, cantidad)
    except (KeyError, ValueError) as e:
        print(f" [Error controlado] producto_id={pid}, cantidad={cantidad} -> {e}")
    else:
        print(f" Venta OK: id_venta={venta['id_venta']}, producto={venta['producto']},
total=${venta['total']}")

print(f"\nTotal de ventas registradas: {len(ventas_buffer)}")
```

Salida obtenida:

```
>>> Registrando ventas de prueba (incluye descuento Reto C)
Venta OK: id_venta=1, producto=Laptop, total=$650.0
Venta OK: id_venta=2, producto=Mouse, total=$77.5
Venta OK: id_venta=3, producto=Teclado, total=$285.0
Venta OK: id_venta=4, producto=Monitor, total=$360.0
Venta OK: id_venta=5, producto=Audífonos, total=$105.0
Venta OK: id_venta=6, producto=Mouse, total=$147.25
Venta OK: id_venta=7, producto=Laptop, total=$1300.0
Venta OK: id_venta=8, producto=Webcam, total=$180.0
Venta OK: id_venta=9, producto=Teclado, total=$75.0
Venta OK: id_venta=10, producto=Monitor, total=$180.0
Venta OK: id_venta=11, producto=Audífonos, total=$498.75
Venta OK: id_venta=12, producto=Mouse, total=$46.5

Total de ventas registradas: 12
```

3.5 Reto D: producto_id inexistente y cantidad inválida

Código:

```
print(">>> Reto D: venta con producto_id inexistente (999) -> se registra en log.txt")
try:
    registrar_venta(999, 1)
except KeyError as e:
    print(f"[Error controlado] {e}")

print("\n>>> Cantidad inválida (0 unidades)")
try:
    registrar_venta(1, 0)
except ValueError as e:
    print(f"[Error controlado] {e}")
```

Salida obtenida:

```
>>> Reto D: venta con producto_id inexistente (999) -> se registra en log.txt
[Error controlado] 'El producto con id 999 no existe en el catálogo.'

>>> Cantidad inválida (0 unidades)
[Error controlado] La cantidad debe ser mayor a 0.
```

3.6 Guardar ventas.csv con Pandas

Código:

```
print(">>> Guardando ventas.csv con Pandas")
df = guardar_ventas_csv()
df
```

Salida obtenida:

```
>>> Guardando ventas.csv con Pandas
   id_venta  producto_id  producto  categoria  cantidad  precio_unitario \
0         1           1    Laptop    Cómputo         1         650.0
1         2           2     Mouse  Accesorios         5          15.5
2         3           3   Teclado  Accesorios        12          25.0
3         4           4   Monitor    Cómputo         2        180.0
4         5           5  Audífonos  Accesorios         3          35.0
5         6           2     Mouse  Accesorios        10          15.5
6         7           1    Laptop    Cómputo         2        650.0
7         8           6   Webcam  Accesorios         4          45.0
8         9           3   Teclado  Accesorios         3          25.0
9        10           4   Monitor    Cómputo         1        180.0
10       11           5  Audífonos  Accesorios        15          35.0
11       12           2     Mouse  Accesorios         3          15.5

   descuento_pct  total  fecha
0           0.00  650.00  2026-08-20 00:01:15
1           0.00  77.50  2026-08-20 00:01:15
2           0.05  285.00  2026-08-20 00:01:15
3           0.00  360.00  2026-08-20 00:01:15
4           0.00  105.00  2026-08-20 00:01:15
5           0.05  147.25  2026-08-20 00:01:15
6           0.00 1300.00  2026-08-20 00:01:15
```

7	0.00	180.00	2026-08-20 00:01:15
8	0.00	75.00	2026-08-20 00:01:15
9	0.00	180.00	2026-08-20 00:01:15
10	0.05	498.75	2026-08-20 00:01:15
11	0.00	46.50	2026-08-20 00:01:15

3.7 Leer ventas.csv de vuelta

Código:

```
print(">>> Leyendo ventas.csv de vuelta")
df_leido = leer_ventas_csv()
df_leido.head(12)
```

Salida obtenida:

```
>>> Leyendo ventas.csv de vuelta
   id_venta  producto_id  producto  categoria  cantidad  precio_unitario \
0         1           1   Laptop    Cómputo           1          650.0
1         2           2   Mouse  Accesorios           5           15.5
2         3           3  Teclado  Accesorios          12           25.0
3         4           4  Monitor    Cómputo           2          180.0
4         5           5  Audífonos  Accesorios           3           35.0
5         6           2   Mouse  Accesorios          10           15.5
6         7           1   Laptop    Cómputo           2          650.0
7         8           6  Webcam  Accesorios           4           45.0
8         9           3  Teclado  Accesorios           3           25.0
9        10           4  Monitor    Cómputo           1          180.0
10       11           5  Audífonos  Accesorios          15           35.0
11       12           2   Mouse  Accesorios           3           15.5

   descuento_pct  total  fecha
0         0.00  650.00  2026-08-20 00:01:15
1         0.00   77.50  2026-08-20 00:01:15
2         0.05  285.00  2026-08-20 00:01:15
3         0.00  360.00  2026-08-20 00:01:15
4         0.00  105.00  2026-08-20 00:01:15
5         0.05  147.25  2026-08-20 00:01:15
6         0.00 1300.00  2026-08-20 00:01:15
7         0.00  180.00  2026-08-20 00:01:15
8         0.00   75.00  2026-08-20 00:01:15
9         0.00  180.00  2026-08-20 00:01:15
10        0.05  498.75  2026-08-20 00:01:15
11        0.00   46.50  2026-08-20 00:01:15
```

3.8 Manejo de error: archivo inexistente

Código:

```
print(">>> Prueba de manejo de error: archivo inexistente")
resultado = leer_ventas_csv("no_existe.csv")
print("Resultado:", resultado)
```

Salida obtenida:

```
>>> Prueba de manejo de error: archivo inexistente
El archivo 'no_existe.csv' no existe todavía. Registra ventas primero.
Resultado: None
```

3.9 Ingresos por producto (Pandas groupby)

Código:

```
print(">>> Ingresos por producto (Pandas groupby)")
resumen = ingresos_por_producto(df_leido)
resumen
```

Salida obtenida:

```
>>> Ingresos por producto (Pandas groupby)
producto
Laptop      1950.00
Audífonos    603.75
Monitor      540.00
Teclado      360.00
Mouse        271.25
Webcam       180.00
Name: total, dtype: float64
```

3.10 Métricas con NumPy

Código:

```
print(">>> Métricas con NumPy")
metricas = calcular_metricas(df_leido)
mostrar_metricas(metricas)
metricas
```

Salida obtenida:

```
>>> Métricas con NumPy

--- MÉTRICAS (NumPy) ---
Ingreso total:      $3905.00
Ingreso promedio/venta: $325.42
Desviación estándar: $343.10
Unidades totales vendidas: 61
Promedio unidades/venta: 5.08
{'ingreso_total': 3905.0,
 'ingreso_promedio': 325.4166666666667,
 'ingreso_desviacion_std': 343.1017270998339,
 'unidades_totales': 61.0,
 'promedio_unidades_por_venta': 5.083333333333333}
```

3.11 Gráfico de ingresos (Reto B)

Código:

```
print(">>> Gráfico de ingresos por producto (Reto B: exportado a ingresos.png)")
graficar_ingresos(df_leido, guardar_png=True, nombre_png="ingresos.png")
```

Salida obtenida:

```
>>> Gráfico de ingresos por producto (Reto B: exportado a ingresos.png)
Gráfico exportado como 'ingresos.png'.
```

3.12 Contenido de log.txt

Código:

```
print(">>> Contenido de log.txt")
with open("log.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

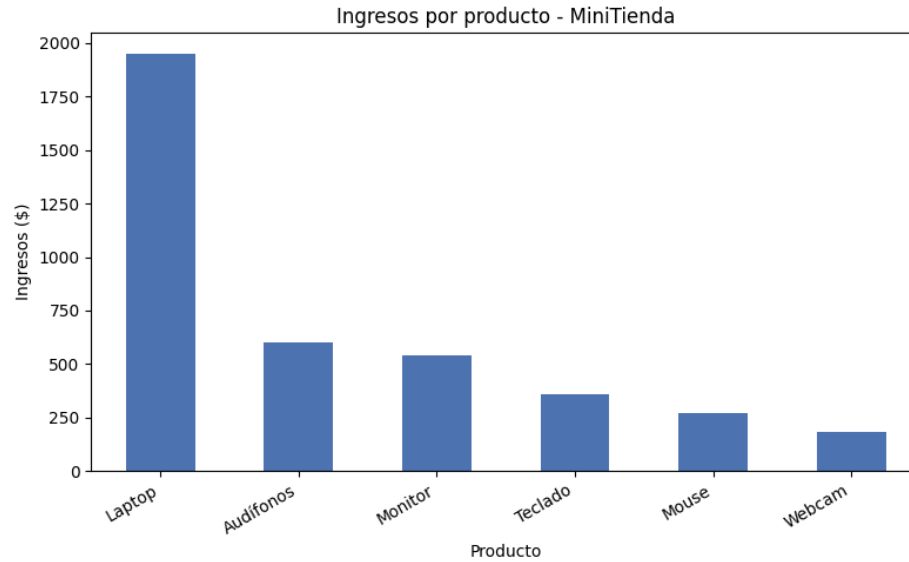
Salida obtenida:

```
>>> Contenido de log.txt
[2026-08-20 00:01:15] Producto nuevo agregado: id=6, nombre=Webcam
[2026-08-20 00:01:15] Venta registrada: {'id_venta': 1, 'producto_id': 1, 'producto':
'Laptop', 'categoria': 'Cómputo', 'cantidad': 1, 'precio_unitario': 650.0,
'descuento_pct': 0.0, 'total': 650.0, 'fecha': '2026-08-20 00:01:15'}
[2026-08-20 00:01:15] Venta registrada: {'id_venta': 2, 'producto_id': 2, 'producto':
'Mouse', 'categoria': 'Accesorios', 'cantidad': 5, 'precio_unitario': 15.5,
'descuento_pct': 0.0, 'total': 77.5, 'fecha': '2026-08-20 00:01:15'}
[2026-08-20 00:01:15] Venta registrada: {'id_venta': 3, 'producto_id': 3, 'producto':
'Teclado', 'categoria': 'Accesorios', 'cantidad': 12, 'precio_unitario': 25.0,
'descuento_pct': 0.05, 'total': 285.0, 'fecha': '2026-08-20 00:01:15'}
[2026-08-20 00:01:15] Venta registrada: {'id_venta': 4, 'producto_id': 4, 'producto':
'Monitor', 'categoria': 'Cómputo', 'cantidad': 2, 'precio_unitario': 180.0,
'descuento_pct': 0.0, 'total': 360.0, 'fecha': '2026-08-20 00:01:15'}
[2026-08-20 00:01:15] Venta registrada: {'id_venta': 5, 'producto_id': 5, 'producto':
'Audífonos', 'categoria': 'Accesorios', 'cantidad': 3, 'precio_unitario': 35.0,
'descuento_pct': 0.0, 'total': 105.0, 'fecha': '2026-08-20 00:01:15'}
[2026-08-20 00:01:15] Venta registrada: {'id_venta': 6, 'producto_id': 2, 'producto':
'Mouse', 'categoria': 'Accesorios', 'cantidad': 10, 'precio_unitario': 15.5,
'descuento_pct': 0.05, 'total': 147.25, 'fecha': '2026-08-20 00:01:15'}
[2026-08-20 00:01:15] Venta registrada: {'id_venta': 7, 'producto_id': 1, 'producto':
'Laptop', 'categoria': 'Cómputo', 'cantidad': 2, 'precio_unitario': 650.0,
'descuento_pct': 0.0, 'total': 1300.0, 'fecha': '2026-08-20 00:01:15'}
[2026-08-20 00:01:15] Venta registrada: {'id_venta': 8, 'producto_id': 6, 'producto':
'Webcam', 'categoria': 'Accesorios', 'cantidad': 4, 'precio_unitario': 45.0,
'descuento_pct': 0.0, 'total': 180.0, 'fecha': '2026-08-20 00:01:15'}
[2026-08-20 00:01:15] Venta registrada: {'id_venta': 9, 'producto_id': 3, 'producto':
'Teclado', 'categoria': 'Accesorios', 'cantidad': 3, 'precio_unitario': 25.0,
'descuento_pct': 0.0, 'total': 75.0, 'fecha': '2026-08-20 00:01:15'}
[2026-08-20 00:01:15] Venta registrada: {'id_venta': 10, 'producto_id': 4, 'producto':
'Monitor', 'categoria': 'Cómputo', 'cantidad': 1, 'precio_unitario': 180.0,
'descuento_pct': 0.0, 'total': 180.0, 'fecha': '2026-08-20 00:01:15'}
[2026-08-20 00:01:15] Venta registrada: {'id_venta': 11, 'producto_id': 5, 'producto':
'Audífonos', 'categoria': 'Accesorios', 'cantidad': 15, 'precio_unitario': 35.0,
'descuento_pct': 0.05, 'total': 498.75, 'fecha': '2026-08-20 00:01:15'}
```

```
[2026-08-20 00:01:15] Venta registrada: {'id_venta': 12, 'producto_id': 2, 'producto':  
'Mouse', 'categoria': 'Accesorios', 'cantidad': 3, 'precio_unitario': 15.5,  
'descuento_pct': 0.0, 'total': 46.5, 'fecha': '2026-08-20 00:01:15'}  
[2026-08-20 00:01:15] INTENTO FALLIDO - producto_id inexistente: 999  
[2026-08-20 00:01:15] CSV guardado: ventas.csv (12 filas)  
[2026-08-20 00:01:15] ERROR - intento de lectura de archivo inexistente: no_existe.csv  
[2026-08-20 00:01:16] Gráfico exportado a PNG: ingresos.png
```

4. Captura del gráfico: ingresos por producto

Este es el gráfico de barras que genero con Matplotlib a partir de ventas.csv, agrupando los ingresos por producto con Pandas (groupby) y exportándolo con plt.savefig('ingresos.png'), que corresponde al Reto B.



5. Respuestas a la asignación

¿Qué parte la hice con Pandas? ¿Qué parte con NumPy?

Con Pandas armé el DataFrame a partir de la lista de ventas, guardé y leí el archivo ventas.csv, y agrupé las ventas por producto con `groupby('producto')['total'].sum()` para obtener los ingresos que después se grafican. Con NumPy hice los cálculos numéricos en sí: `np.sum`, `np.mean` y `np.std` sobre los arreglos de ingresos y cantidades, que me dan el ingreso total, el promedio por venta y la desviación estándar.

¿Dónde usé try/except y por qué?

Lo usé en tres partes principales: en `pedir_entero()`, para capturar el error si el usuario escribe algo que no se puede convertir a número; en el menú, alrededor de `registrar_venta()`, para capturar cuando el producto no existe o la cantidad o el stock no son válidos, sin que el programa se detenga; y en `leer_ventas_csv()`, para capturar el error si el archivo ventas.csv todavía no existe. En el registro de venta usé el `try/except/else/finally` completo: el `else` corre solo si la venta se registró sin errores, y el `finally` siempre imprime un mensaje de cierre al final del intento.

¿Qué estructuras son tuplas, listas y diccionarios en el código?

Tuplas: cada producto individual del catálogo, con la forma (id, nombre, categoría). Listas: `catalogo`, que es la lista de tuplas de productos, y `ventas_buffer`, la lista de diccionarios donde se va guardando cada venta registrada. Diccionarios: `precios` y `stock`, con el `id_producto` como clave, y también cada venta individual dentro de `ventas_buffer`, que se guarda como un diccionario con sus propios campos (`id_venta`, `producto`, `cantidad`, `total`, etc.).